

OS Lab 5

Inter-Process Communication

Pipe System Call

pipe() is a Linux system function. The **pipe()** system function is used to open file descriptors, which are used to communicate between different Linux processes. In short, the **pipe()** function is used for inter-process communication in Linux.

The syntax of the **pipe()** function is:

```
int pipe(int pipefd[2]);
```

The **pipe()** function creates a unidirectional data channel for inter-process communication. You pass in an **int** (Integer) type array **pipefd** consisting of 2 array element to the function **pipe()**. Then the **pipe()** function creates two file descriptors in the **pipefd** array. The first element of the **pipefd** array, **pipefd[0]** is used for reading data from the pipe. The second element of the **pipefd** array, **pipefd[1]** is used for writing data to the pipe. On success, the **pipe()** function returns 0. If an error occurs during pipe initialization, then the **pipe()** function returns -1.

The **pipe()** function is defined in the header **unistd.h**. In order to use the **pipe()** function in your C program, you must include the header **unistd.h** as follows:

```
#include <unistd.h>
```

For more information on the **pipe()** system function, check the man page of **pipe()** with the following command:

```
$ man 2 pipe  
The man page of pipe().
```

Example 1:

For the first example, create a new C source file **1_pipe.c** and type in the following lines of codes.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int main(void) {  
    int pipefds[2];  
  
    if(pipe(pipefds) == -1) {  
        perror("pipe");  
        exit(EXIT_FAILURE);  
    }
```

```

}

printf("Read File Descriptor Value: %d\n", pipefds[0]);
printf("Write File Descriptor Value: %d\n", pipefds[1]);

return EXIT_SUCCESS;
}

```

When you run the program, you should see the following output. As you can see, the value of the read pipe file descriptor **pipefds[0]** is **3** and write pipe file descriptor **pipefds[1]** is **4**.

```

system-programming : bash — Konsole
File Edit View Bookmarks Settings Help
shovon@dev-vm2:~/Projects/system-programming
$ gcc 1_pipe.c && ./a.out
Read File Descriptor Value: 3
Write File Descriptor Value: 4
shovon@dev-vm2:~/Projects/system-programming
$ 

```

Example 2:

Create another C source file **2_pipe.c** and type in the following lines of codes.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    int pipefds[2];
    char buffer[5];

    if(pipe(pipefds) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    char *pin = "4128\0";

    printf("Writing PIN to pipe...\n");
    write(pipefds[1], pin, 5);
    printf("Done.\n\n");

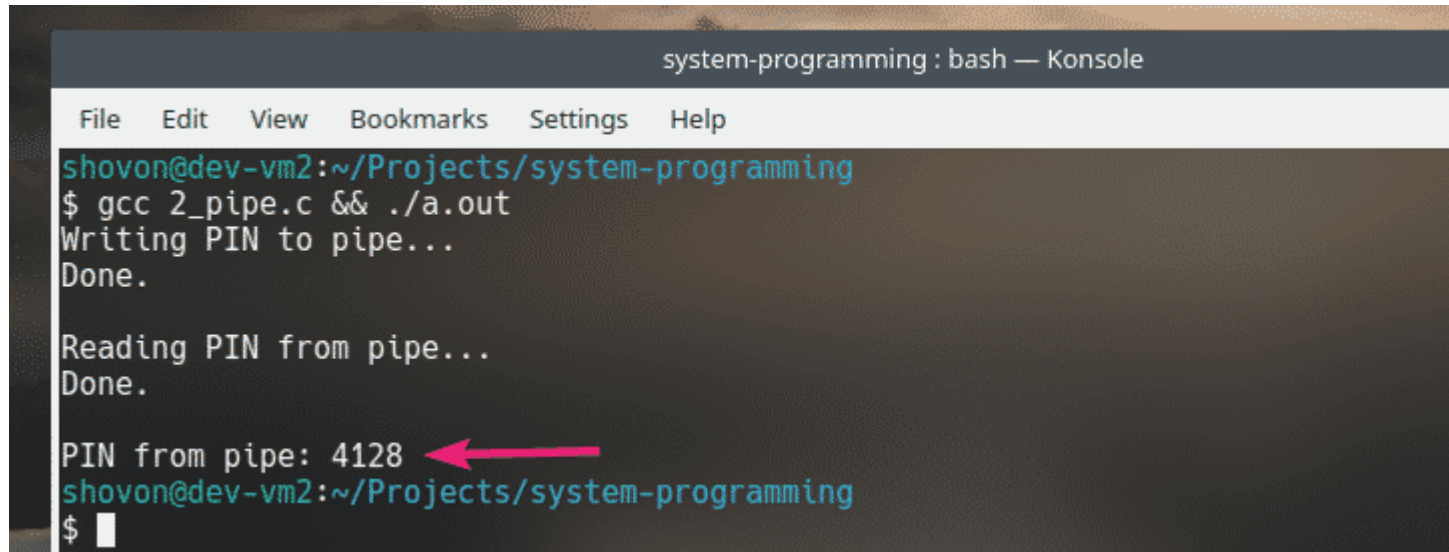
    printf("Reading PIN from pipe...\n");
    read(pipefds[0], buffer, 5);
    printf("Done.\n\n");
}

```

```
printf("PIN from pipe: %s\n", buffer);

return EXIT_SUCCESS;
}
```

When you run the program, the correct output is displayed as you can see.



```
system-programming : bash — Konsole
File Edit View Bookmarks Settings Help
shovon@dev-vm2:~/Projects/system-programming
$ gcc 2_pipe.c && ./a.out
Writing PIN to pipe...
Done.

Reading PIN from pipe...
Done.

PIN from pipe: 4128
shovon@dev-vm2:~/Projects/system-programming
$
```

Example 3:

Create a new C source file **3_pipe.c** as type in the following lines of codes.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(void) {
    int pipefds[2];
    char *pin;
    char buffer[5];

    if(pipe(pipefds) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid_t pid = fork();

    if(pid == 0) { // in child process
        pin = "4821\0"; // PIN to send
        close(pipefds[0]); // close read fd
        write(pipefds[1], pin, 5); // write PIN to pipe
    }
}
```

```

printf("Generating PIN in child and sending to parent...\n");
sleep(2); // intentional delay
exit(EXIT_SUCCESS);
}

if(pid > 0) { // in main process
    wait(NULL); // wait for child process to finish
    close(pipefds[1]); // close write fd
    read(pipefds[0], buffer, 5); // read PIN from pipe
    close(pipefds[0]); // close read fd

    printf("Parent received PIN '%s'\n", buffer);
}

return EXIT_SUCCESS;
}

```

As you can see, running the program gives the expected result.

```

system-programming : bash — Konsole
File Edit View Bookmarks Settings Help
shovon@dev-vm2:~/Projects/system-programming
$ gcc 3_pipe.c && ./a.out
Generating PIN in child and sending to parent...
Parent received PIN '4821'
shovon@dev-vm2:~/Projects/system-programming
$

```

Example 4:

Create a new C source file **4_pipe.c** as type in the following lines of codes.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define PIN_LENGTH 4
#define PIN_WAIT_INTERVAL 2

void getPIN(char pin[PIN_LENGTH + 1]) {
    srand(getpid() + getppid());

    pin[0] = 49 + rand() % 7;

    for(int i = 1; i < PIN_LENGTH; i++) {

```

```

    pin[i] = 48 + rand() % 7;
}

pin[PIN_LENGTH] = '\0';
}

int main(void) {
    while(1) {
        int pipefds[2];
        char pin[PIN_LENGTH + 1];
        char buffer[PIN_LENGTH + 1];

        pipe(pipefds);

        pid_t pid = fork();

        if(pid == 0) {
            getPIN(pin); // generate PIN
            close(pipefds[0]); // close read fd
            write(pipefds[1], pin, PIN_LENGTH + 1); // write PIN to pipe

            printf("Generating PIN in child and sending to parent...\n");

            sleep(PIN_WAIT_INTERVAL); // delaying PIN generation intentionally.

            exit(EXIT_SUCCESS);
        }

        if(pid > 0) {
            wait(NULL); // waiting for child to finish

            close(pipefds[1]); // close write fd
            read(pipefds[0], buffer, PIN_LENGTH + 1); // read PIN from pipe
            close(pipefds[0]); // close read fd
            printf("Parent received PIN '%s' from child.\n\n", buffer);
        }
    }

    return EXIT_SUCCESS;
}

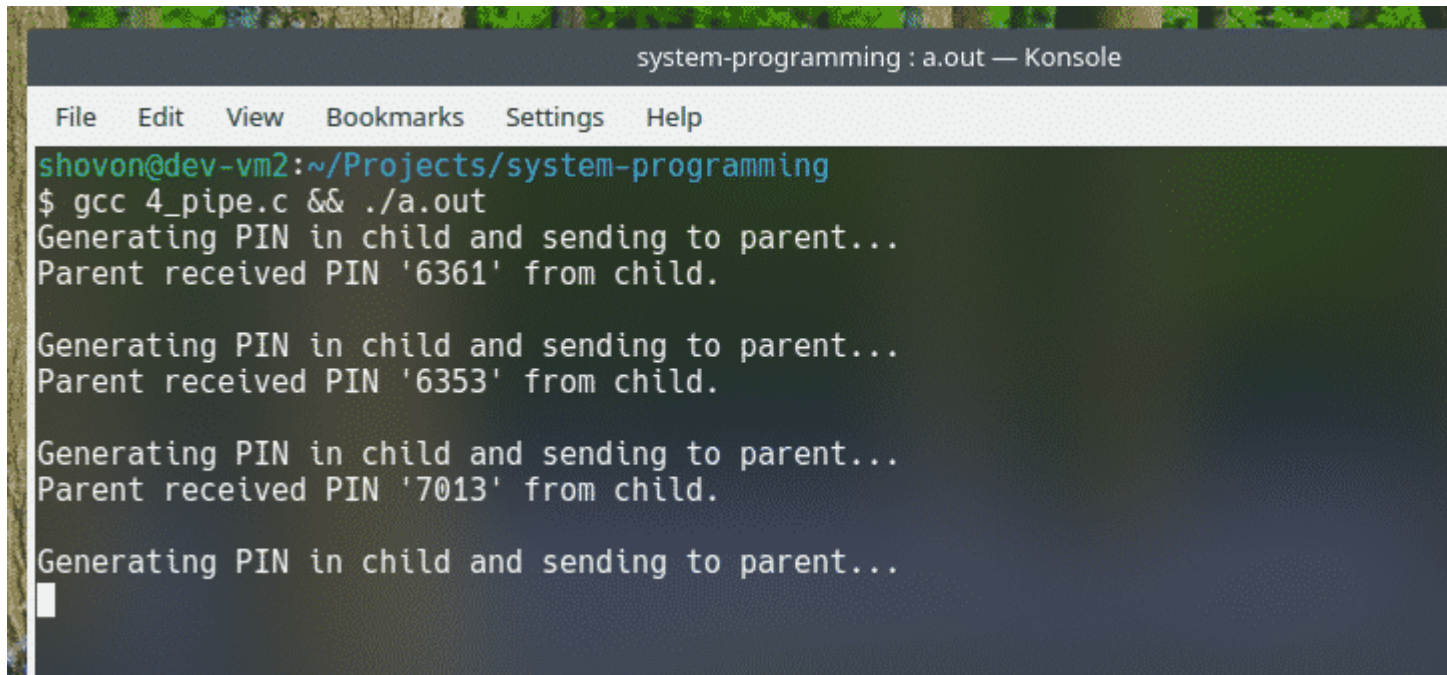
```

This example is the same as **Example 3**. The only difference is that this program continuously creates a child process, generates a PIN in the child process and sends the PIN to the parent process using a pipe.

The parent process then reads the PIN from the pipe and prints it.

This program generates a new PIN_LENGTH PIN every PIN_WAIT_INTERVAL seconds.

As you can see, the program works as expected.



```
system-programming : a.out — Konsole
File Edit View Bookmarks Settings Help
shovon@dev-vm2:~/Projects/system-programming
$ gcc 4_pipe.c && ./a.out
Generating PIN in child and sending to parent...
Parent received PIN '6361' from child.

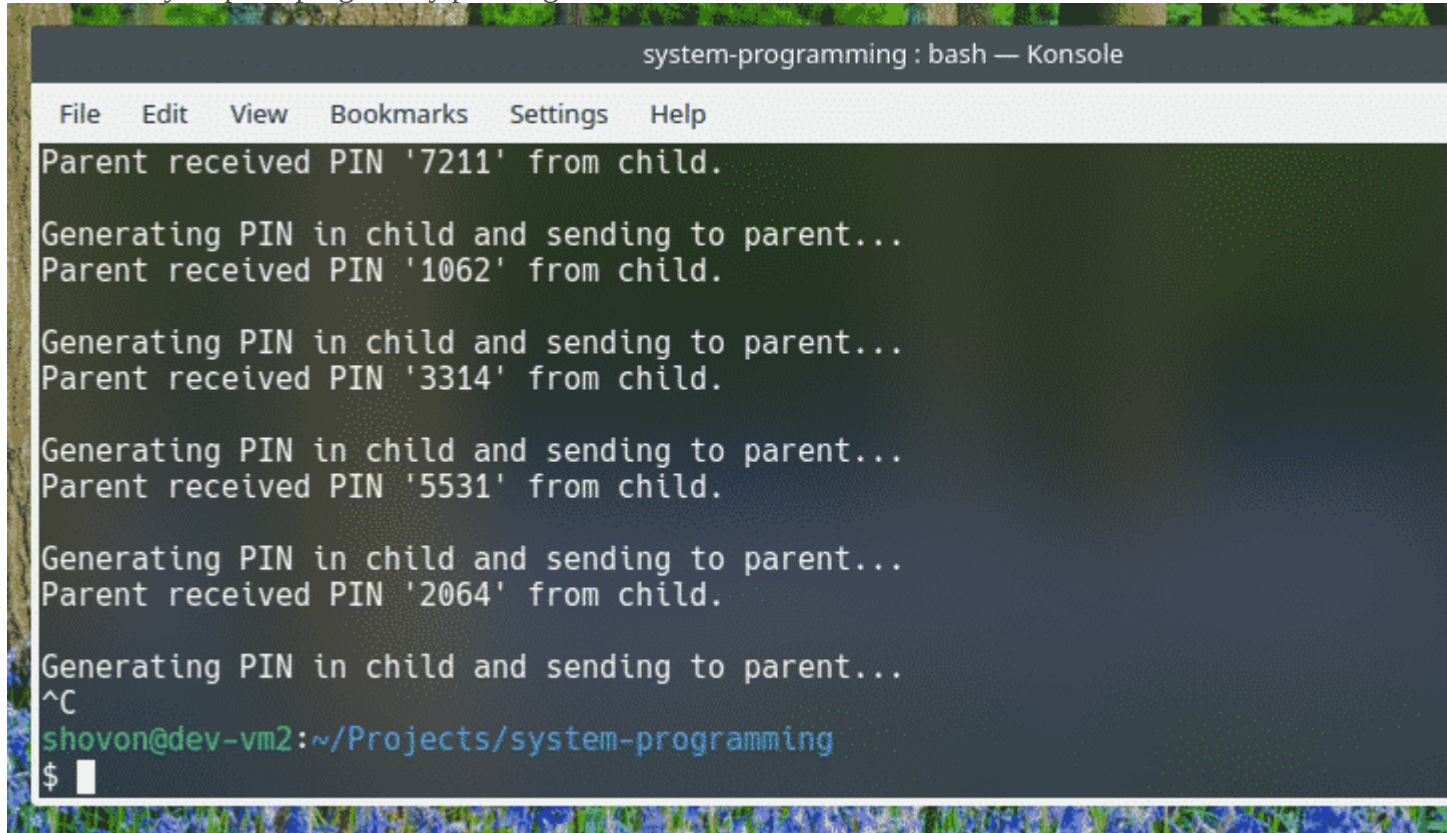
Generating PIN in child and sending to parent...
Parent received PIN '6353' from child.

Generating PIN in child and sending to parent...
Parent received PIN '7013' from child.

Generating PIN in child and sending to parent...

```

You can only stop the program by pressing **<Ctrl> + C**.



```
system-programming : bash — Konsole
File Edit View Bookmarks Settings Help
Parent received PIN '7211' from child.

Generating PIN in child and sending to parent...
Parent received PIN '1062' from child.

Generating PIN in child and sending to parent...
Parent received PIN '3314' from child.

Generating PIN in child and sending to parent...
Parent received PIN '5531' from child.

Generating PIN in child and sending to parent...
Parent received PIN '2064' from child.

Generating PIN in child and sending to parent...
^C
shovon@dev-vm2:~/Projects/system-programming
$ 
```

So, this is how you use the `pipe()` system call in C programming language.